

Experiment 8

Module: Unit Test Frameworks Experiment: Data-Driven Automation (DDT)

Problem statement. Build a login test that reads multiple cases from an external CSV file and validates the expected result for each row.

Objective. Separate test data from execution logic so new login scenarios can be added without changing the test code.

Tools. Python 3, Selenium WebDriver, the csv module, and explicit waits.

1 Project structure

```
selenium_ddt/  
|-- test_data/  
|   |-- credentials.csv  
|-- tests/  
|   |-- test_login_ddt.py
```

2 Create the test data

Save this as test_data/credentials.csv. Quoted fields keep long error messages intact.

```
username,password,expected_error  
standard_user,secret_sauce,None  
locked_out_user,secret_sauce,"Epic sadface: Sorry, this user has been locked out."  
invalid_user,wrong_pass,"Epic sadface: Username and password do not match any user in this service"
```

credentials.csv - test matrix

username	password	expected_error
standard_user	secret_sauce	None
locked_out_user	secret_sauce	Epic sadface: Sorry, this user has been locked out.
invalid_user	wrong_pass	Epic sadface: Username and password do not match any user in this service

Evidence screenshot 1: external test matrix.

3 Implement the data driven test

```
from csv import DictReader
from pathlib import Path
import time

from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support import expected_conditions as EC
from selenium.webdriver.support.ui import WebDriverWait

ROOT = Path(__file__).resolve().parents[1]
CSV_FILE = ROOT / "test_data" / "credentials.csv"

def read_cases(path: Path):
    with path.open(newline="", encoding="utf-8") as file:
        return list(DictReader(file))

for case in read_cases(CSV_FILE):
    driver = webdriver.Chrome()
    wait = WebDriverWait(driver, 10)
    try:
        driver.get("https://www.saucedemo.com/")
        wait.until(EC.visibility_of_element_located((By.ID, "user-name"))).send_keys(case["username"])
        driver.find_element(By.ID, "password").send_keys(case["password"])
        driver.find_element(By.ID, "login-button").click()

        if case["expected_error"] == "None":
            wait.until(EC.url_contains("inventory.html"))
            print(f"PASS: Valid login worked for {case['username']}")
        else:
            error = wait.until(
                EC.visibility_of_element_located((By.CSS_SELECTOR, "h3[data-test='error']"))
            ).text
            assert case["expected_error"] in error, f"Expected: {case['expected_error']} | Actual: {error}"
            print(f"PASS: Correct error caught for {case['username']}")
    finally:
        time.sleep(3)
        driver.quit()
```

4 Run the experiment

```
python -m pip install -r requirements.txt
```

```
python tests/test_login_ddt.py
```

Evidence screenshot 2: execution output after all three rows.

Swag Labs

locked_out_user

Epic sadface: Sorry, this user has been locked out.

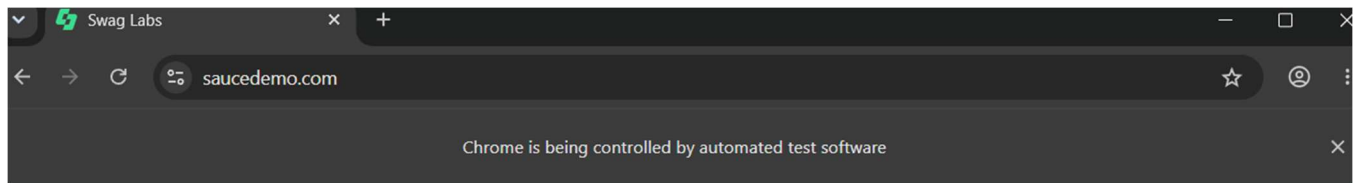
Login

Accepted usernames are:

standard_user
locked_out_user
problem_user
performance_glitch_user
error_user
visual_user

Password for all users:

secret_sauce



Swag Labs

invalid_user ✖

..... ✖

Epic sadrace: Username and password do not match any user in this service ✖

Login

Accepted usernames are:

standard_user
locked_out_user
problem_user
performance_glitch_user
error_user
visual_user

Password for all users:

secret_sauce

Problems Output Debug Console Terminal Ports

```
> python -u "c:\Users\rahul\Documents\Codex\2026-09-21\can-you-rewrite-this-pdf-and-4\selenium_ddt\tests\tempCodeRunnerFile.py"
PASS: Valid login worked for standard_user
PASS: Correct error caught for locked_out_user
PASS: Correct error caught for invalid_user
PS C:\Users\rahul\Documents\Codex\2026-09-21\can-you-rewrite-this-pdf-and-4\selenium_ddt>
```

5 Verified results

Case	Expected behavior	Observed result	Status
standard_user	Redirect to inventory.html	Products page opened	PASS
locked_out_user	Locked-out error	Exact locked-out message displayed	PASS
invalid_user	Credential mismatch error	Exact mismatch message displayed	PASS

Conclusion. The implementation separates data from automation logic and validates three login outcomes from one CSV-driven loop. The live run matched all three expected outcomes.